T.C

ISPARTA UYGULAMALI BİLİMLER ÜNİVERSİTESİ

TEKNOLOJİ FAKÜLTESİ

ELEKTRİK-ELEKTRONİK MÜHENDİSLİĞİ

MAKİNE ÖĞRENMESİNE GİRİŞ DERSİ OBJECT DETECTİON (Modern Yaklaşım)
SUNUM RAPORU UMUT KAAN BOSTANCI 2112705054

1-YOLO

Yolo makine öğrenmesi metodlarında çoğu insan tarafından bir devrim olarak kabul görür.Yolo anlık nesne tahmini yaparak saniyeler içerisinde görseldeki nesneleri büyük bir oranda doğru bir şekilde tahmin edip gösterebilir.Yolo verilen görseli ızgaralar haline bölerek görseli parçalar ve böldüğü ızgaralarda tahminler yaparak nesneyi işaretleyerek doğru sonucu yüksek oranda bulur.

Aşağıdaki görselde örnek bir YOLO v8 kodlaması verilmiştir

h_python.py 2

h_python.py > ...

```python
from ultralytics import YOLO
import cv2

# 1. Adım: Önceden eğitilmiş YOLOv8 modelini yükle.
# 'yolov8n.pt' (nano) modeli çok hızlıdır ve hafif uygulamalar için idealdir.
# Model dosyası bilgisayarınızda yoksa kod çalışırken otomatik olarak indirilir.
model = YOLO("yolov8n.pt")

# 2. Adım: Tespit yapılacak görüntünün yolunu belirle.
# Buraya bilgisayarınızdaki bir resmin yolunu yazmalısınız.
# (Örnek olarak internetten indirdiğiniz bir araba veya insan fotoğrafı kullanabilirsiniz)
image_path = "ornek_resim.jpg"

# 3. Adım: Modeli görüntü üzerinde çalıştır (Inference).
# save=True parametresi, tespit edilen nesnelerin işaretlendiği yeni bir resim kaydeder.
results = model(image_path, save=True)

# 4. Adım: Sonuçları konsola yazdır veya incele
for result in results:
    boxes = result.boxes  # Tespit edilen nesnelerin sınır kutuları (bounding boxes)

    print("\n--- Tespit Edilen Nesneler ---")
    for box in boxes:
        # Sınıf ID'sini al ve modelin sınıf isimleri sözlüğünden (names) adını bul
        class_id = int(box.cls[0])
        class_name = model.names[class_id]

        # Güven skorunu (confidence) al (% kaç emin olduğu)
        confidence = round(float(box.conf[0]), 2)

        print(f"Nesne: {class_name} | Güven Skoru: {confidence}")

print("\nİşlem tamamlandı! Çıktı görüntüsü 'runs/detect/predict' klasörüne kaydedildi.")
```

2-Intersection over Union (IoU)

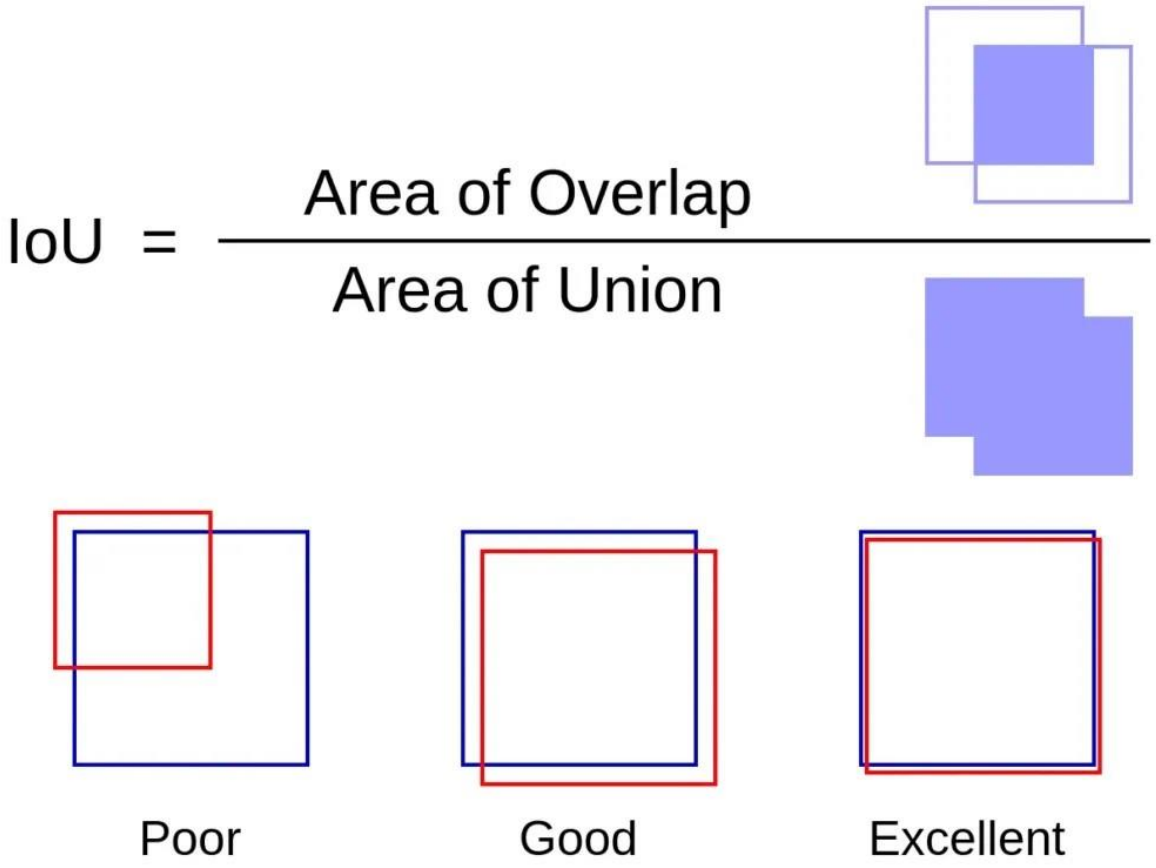
IoU tahmin edilen görsel kutusu ile gerçekte olması gereken kutunun ne kadar örtüştüğünü ölçer.

IoU formulü : kutuların kesişim alanı / kutuların birleşim alanı olarak ifade edilir.

IoU değeri 0-1 arasındadır

ÖRNEĞİN bir algoritmada IoU eşik değeri 0.5 verilirse bu değerin üstündeki tahminler doğru tahmin olarak değerlendirilir.

Aşağıdaki görselde IoU değeri hakkında çerçevelerle bilgilendirilme yapılmıştır.



3- Non-Max Suppression (NMS)

Non-Max Suppression (NMS) aynı nesne için birden fazla yapılan tahmini doğruya en yakın olacak şekilde yani IoU değeri en yüksek olacak şekilde seçme ve doğruya daha az yakın olan seçimleri kaldırma işlemini yaparak tahmini tek bir kutuya indirir. Yalnız görselde birden fazla nesne için yapılan birden fazla tahmini bir insanın gözü gibi ayıramadığı için belli bir algoritma kullanır.

Bu algoritma aşağıdaki görselde verilmiştir

Non-max suppression algoritması

19× 19

Her çıktı tahmini : $\begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \end{bmatrix}$

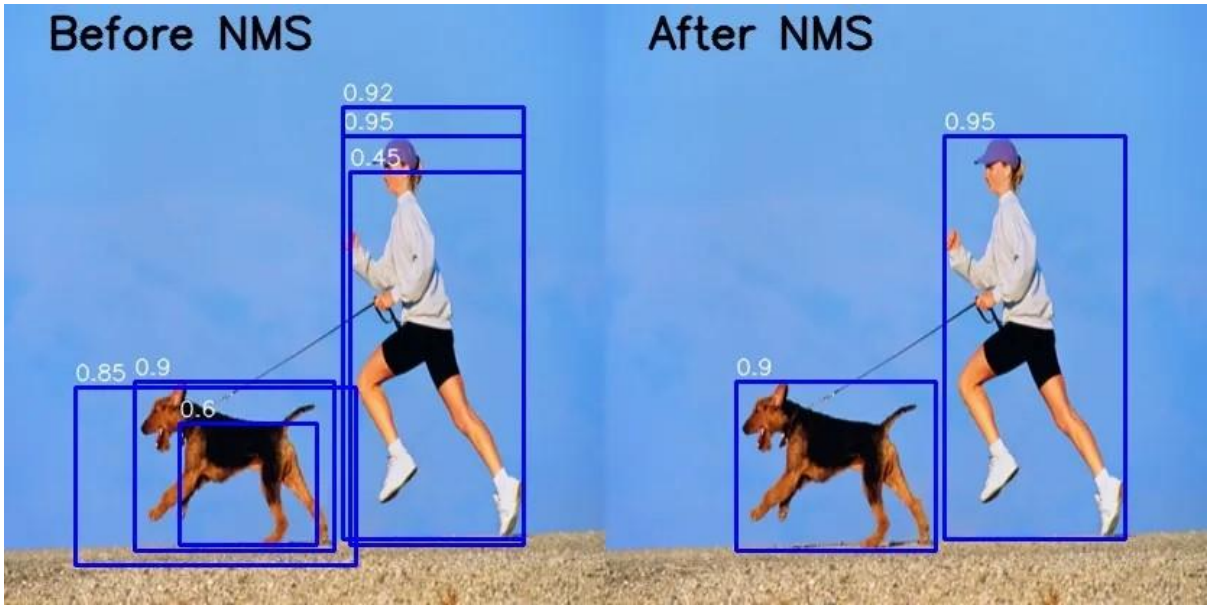
c_1
c_2
c_3

$p_c \leq 0.6$ olan kutuları at.

Kalan kutular olduğu sürece:

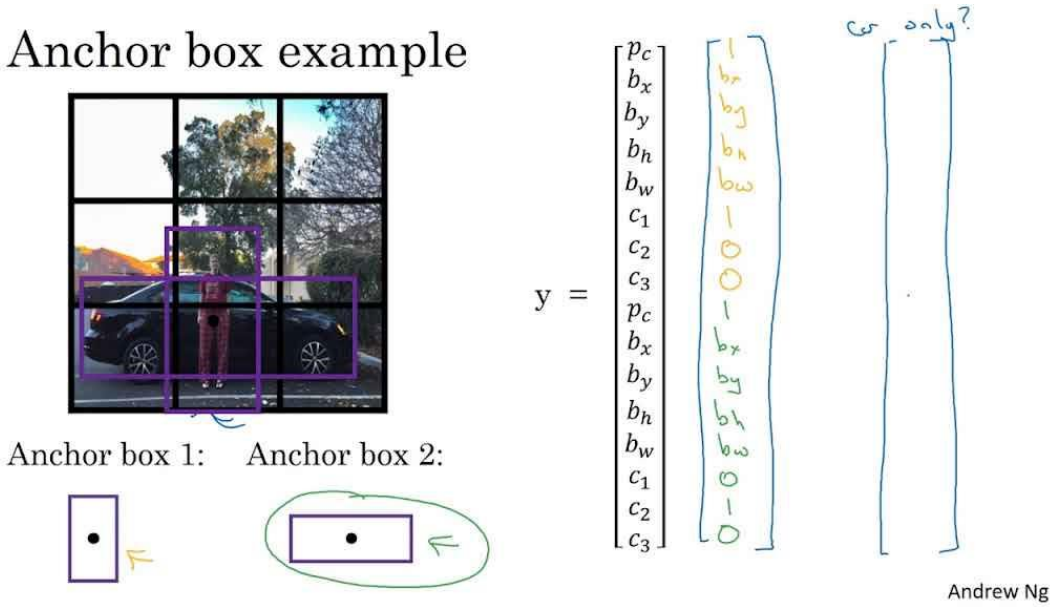
- En büyük p_c'ye sahip kutuyu seç
  Bunu bir tahmin olarak kabul et.
- Önceki adımda kutu çıktısı ile IoU ≥0,5 olan kutuları at.

Andrew Ng



Yukarıdaki görselde NMS öncesi ve NMS sonrası nesne tahmin kutucukları verilmiştir

4- Anchor Boxes



Farklı boyut ve orandaki nesneleri yakalama amacıyla önceden hazırlanmış kutulardır.

Her ızgara hücresine önceden bu kutulardan bir kaç tane tanımlanır. Model bu kutuları nesneye göre ayarlar. Önemli olma sebebi bir görseldeki nesnelerin boyutları farklı olabilir örneğin yatay halde bulunan bir araba ile dikey halde bulunan bir insan aynı görselin aynı ızgarasında denk gelebilir ve bunları yakalamak için farklı tarzda kutular kullanmak gerekir. Yukarıdaki örnekte bu mevcuttur.

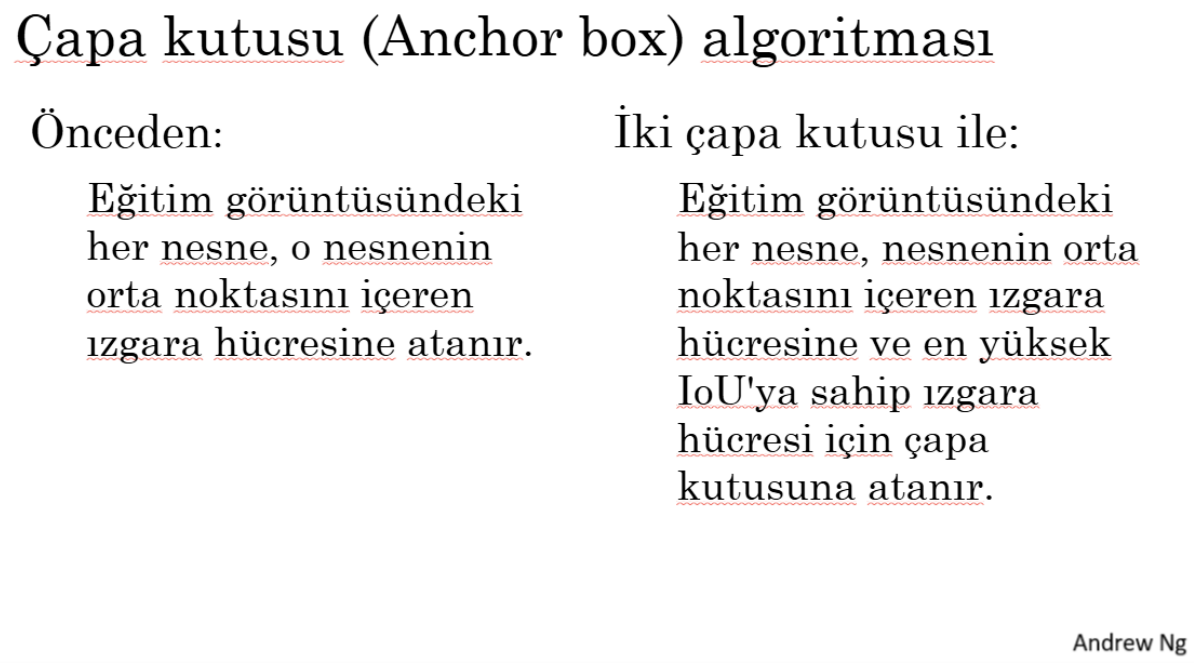
```
# parameters
nc: 80  # number of classes
depth_multiple: 0.33  # model depth multiple
width_multiple: 0.50  # layer channel multiple

# anchors
anchors:
  - [116,90, 156,198, 373,326]  # P5/32
  - [30,61, 62,45, 59,119]  # P4/16
  - [10,13, 16,30, 33,23]  # P3/8

# YOLOv5 backbone
```

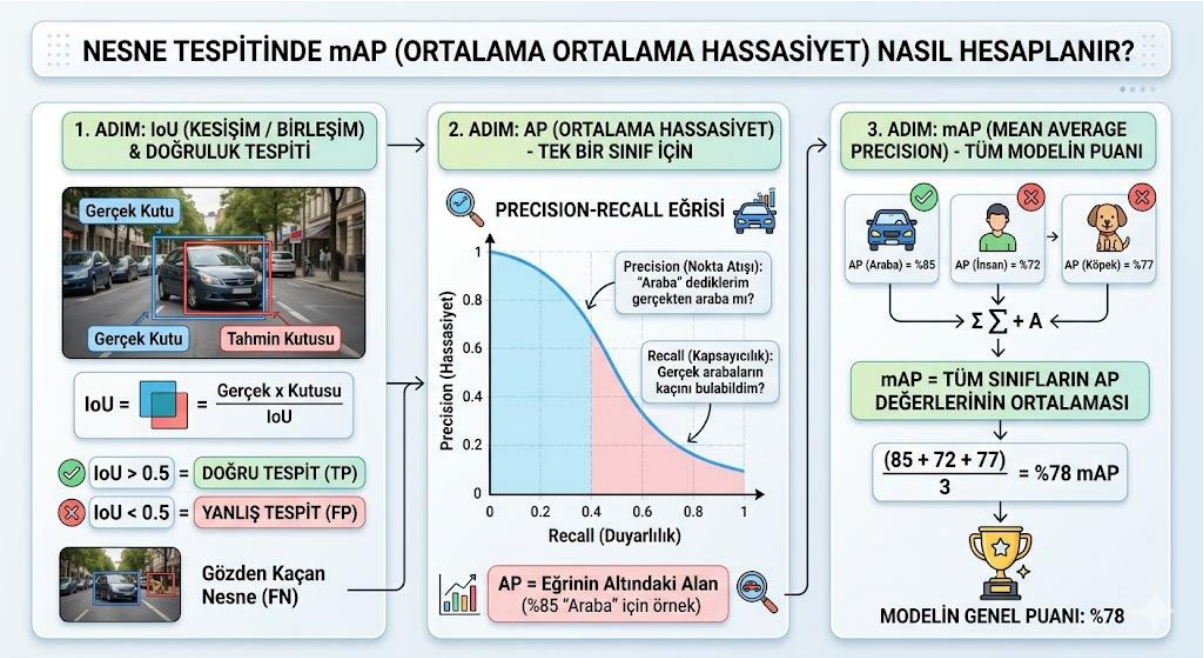
YOLOv5 te bu kutular yukarıdaki görseldeki şekilde yapılandırılmaktadır.

Aşağıdaki fotoğrafta ise algoritmasından bahsetmektedir.

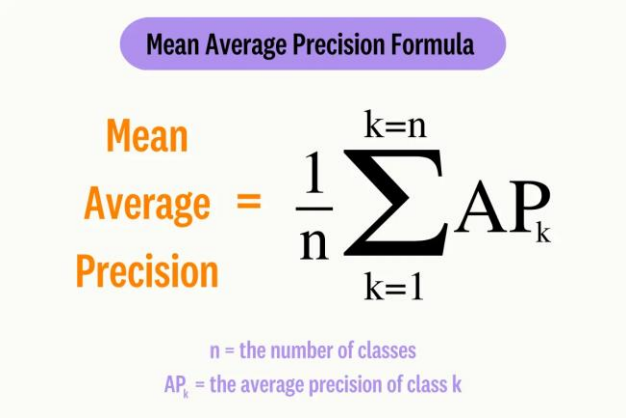


5- mAP (mean Average Precision)

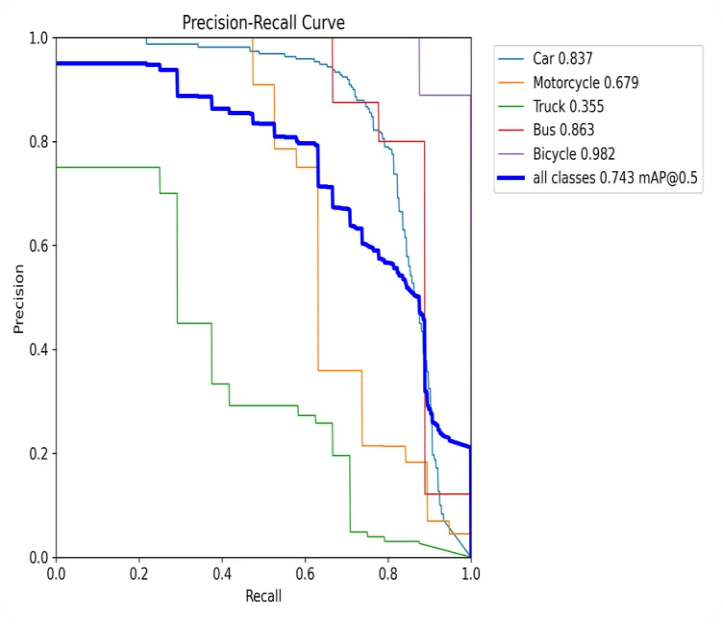
MAP tüm sistemin ortalama doğruluk hassasiyeti algoritmasıdır. Her bir gridin IoU değerine göre doğruluk hassasiyeti ortalaması bulunduktan sonra alınan hepsinin ortalamasına mAP denir



Yukarıdaki görselde mAP değerinin nasıl hesaplandığı yapay zeka tarafından oluşturulmuştur.



Yandaki görselde mAP değerinin formulize hali verilmiştir.



Bu görselde ise hassasiyet grafiğine yer verilmiştir. Mavi çizgimiz mAP değerini temsil etmektedir.

YAPMIŞ OLDUĞUM MİNİ PROJE HAKKINDA

Bu projenin temel amacı, bir yapay zeka modelinin tahminlerine ne kadar **"güvenmesi"** gerektiğini belirleyen **Threshold (Güven Eşiği)** kavramının, sonuçların doğruluğu ve miktarı üzerindeki etkisini deneysel olarak gözlemlemektir.

Bunu biraz daha somutlaştırırsak, proje şu üç ana hedefi gerçekleştirir:

1. Karar Mekanizmasını Anlamak

Yapay zeka modelleri "Bu bir köpektir" demez. Aslında "Bu görüntünün köpek olma ihtimali %94'tür" der. Projenin amacı, bu yüzde kaçın üzerindeki sonuçları **"doğru kabul edeceğimize"** karar verme sürecini simüle etmektir.

2. Hassasiyet ve Kesinlik Dengesi (Precision vs. Recall)

Proje size şu dengeyi öğretir:

- **Düşük Eşik (Örn: %10):** "En ufak bir benzerlik görsen bile bana söyle." Bu durumda model çok fazla sonuç bulur ama hata yapma (yanlış alarm) ihtimali artar.
- **Yüksek Eşik (Örn: %90):** "Sadece adın gibi eminsen bana söyle." Bu durumda model sadece çok net sonuçları verir, hata payı azalır ama bazı doğru nesneleri de gözden kaçırabilir.

3. Filtreleme Mantığını Kavramak

Gerçek dünya uygulamalarında (örneğin otonom araçlarda) modelin her gördüğü karaltıyı "yaya" sanıp fren yapmaması gerekir. Bu proje sayesinde, karmaşık ham veriden (raw predictions) anlamlı ve güvenilir veriye nasıl geçiş yapıldığını, kod seviyesinde deneyimlemiş olursunuz.

Özetle: Bu bir **"Kalite Kontrol"** projesidir. Modelin performansını değil, modelin sonuçlarını kullanırken sizin ne kadar seçici olmanız gerektiğini analiz eder.

SONUÇ ve Öğrendiklerim

1. Teoriden Pratiğe Köprü Kurdum

Sadece ders kitaplarındaki RLC devrelerini veya nodal analiz denklemlerini çözmekle kalmadım; bu teorik bilgilerin bir AC-DC dönüştürücü tasarlarken **güç verimliliği ve sinyal kararlılığı** için ne kadar kritik olduğunu anladım. Fazör

diyagramlarının sadece kağıt üzerinde kalmadığını, gerçek dünyadaki empedans hesaplarının temeli olduğunu kavradım.

2. Üretim Standartlarını Benimsedim

KiCad kullanarak bir devreyi sadece "çizmedim", onu üretime hazır hale getirmeyi öğrendim. Gerber katmanlarını PDF olarak dökümante etme ihtiyacım, mühendisliğin sadece tasarlamak değil, aynı zamanda **standartlara uygun iş teslim etmek** olduğunu gösterdi. Bir trafoyu modellemekten kart dizgisine kadar olan süreçte disiplin kazandım.

3. Yapay Zeka ile Görsel Zekayı Entegre Ettim

Elektronik dünyasından bir adım öteye geçerek, bir sistemin "gözü" sayılan YOLO algoritmasına giriş yaptım. Nesne tespitinin sadece bir kodu çalıştırmak olmadığını; hız, doğruluk ve optimizasyon (YOLOv8, YOLOv10 gibi) arasındaki o ince dengeyi keşfettim.

4. Başarıyı Ölçmeyi Öğrendim

"Sistem çalışıyor" demenin bir mühendis için yeterli olmadığını, mAP (Mean Average Precision) ve IoU gibi metrikleri inceleyerek öğrendim. Bir modelin başarısını **istatistiksel ve matematiksel verilerle kanıtlamanın**, projenin güvenilirliği için en önemli adım olduğunu fark ettim.

Temel Çıkarımım

Donanımı (devre analizi ve PCB) ve yazılımı (YOLO ve Python) birbirinden kopuk iki ayrı dünya olarak değil, birbirini besleyen bir bütün olarak görmeye başladım. Kendi tasarladığım bir PCB üzerinde koşan, nesne tanıyan ve kararlar verebilen **"akıllı sistemler"** inşa edebilecek vizyona ulaştım. Artık bir projeye bakarken sadece kabloları değil, o kabloların taşıdığı verinin nasıl bir zekaya dönüşebileceğini de görüyorum.

KAYNAKÇA:

[1] A. Şentürk, "EEM272 Devre Analizi II Ders Notları: 07_slayt," *asenturk.github.io*. [Çevrimiçi]. Erişilebilir: https://github.com/asenturk/asenturk.github.io/blob/main/eem272/dersler/images/07_slayt.pptx

[2] "YOLOv8 Tutorial: Real-Time Object Detection," *YouTube*, 2024. [Video]. Erişilebilir: https://www.youtube.com/watch?v=VAo84c1hQX8&t=84s

[3] "Deep Learning and Computer Vision Insights," *YouTube*, 2024. [Video]. Erişilebilir: https://www.youtube.com/watch?v=RgQbweTwrkU

[4] Deep Learning Türkiye, "YOLO Algoritmasını Anlamak," *Medium*, 2018. [Çevrimiçi]. Erişilebilir: https://medium.com/deep-learning-turkiye/yolo-algoritmas%C4%B1n%C4%B1-anlamak-290f2152808f